

Build a Database Server

Objective

The main goal of this lab was to set up a managed relational database using Amazon RDS. I created a secure MySQL database with high availability, allowed only the web server to connect to it, and then used a web application to add, change, and delete contacts to check that everything was working properly. **Key AWS services and concepts covered:**

- Amazon Relational Database Service (RDS)
- Security groups to control who can connect
- DB Subnet Groups for placing the database in different zones
- Multi-AZ setup for better reliability
- Connecting a web app to the database using endpoint and login details

Scenario

The lab already had a VPC called Lab VPC with a web server running in a public subnet. I had to add a MySQL database using RDS, make it run across two Availability Zones so it would be more reliable, allow only the web server to reach it, and then test it by using an Address Book web application to save and manage contact information.

Lab Architecture Overview

The setup had a web server in a public subnet talking to a Multi-AZ RDS MySQL database in private subnets across two Availability Zones.

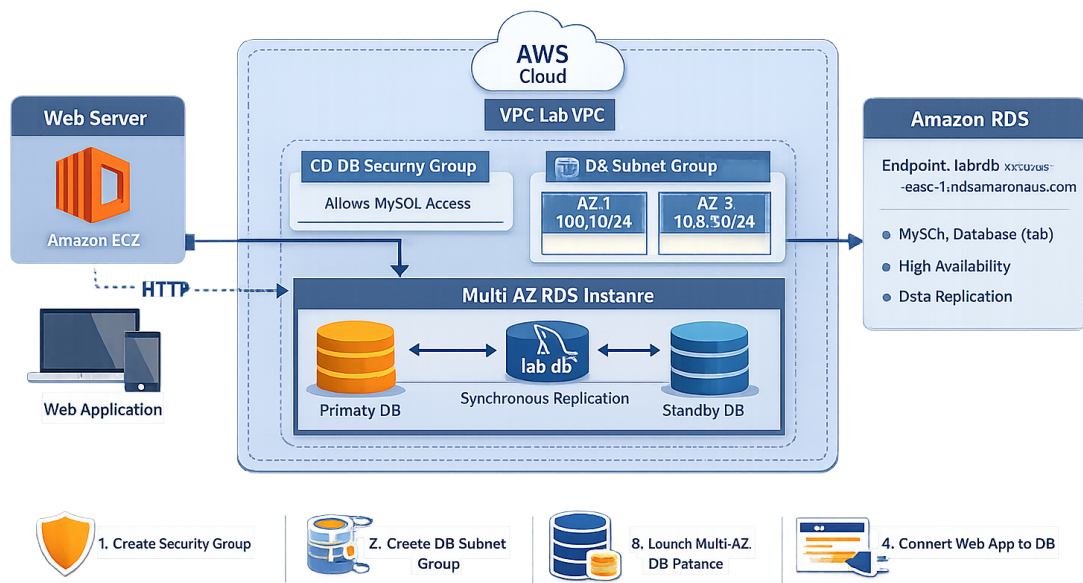


Figure 1: Web server connected to Multi-AZ RDS MySQL database in the Lab VPC

This picture made it easier to see how the database stays hidden from the internet but still works with the web app.

Working Methodology

Task 1: Creating a Security Group for the RDS DB Instance

I went to the VPC console and made a new security group called DB Security Group in the Lab VPC. I added a rule to allow MySQL traffic (port 3306) only from the Web Security Group.

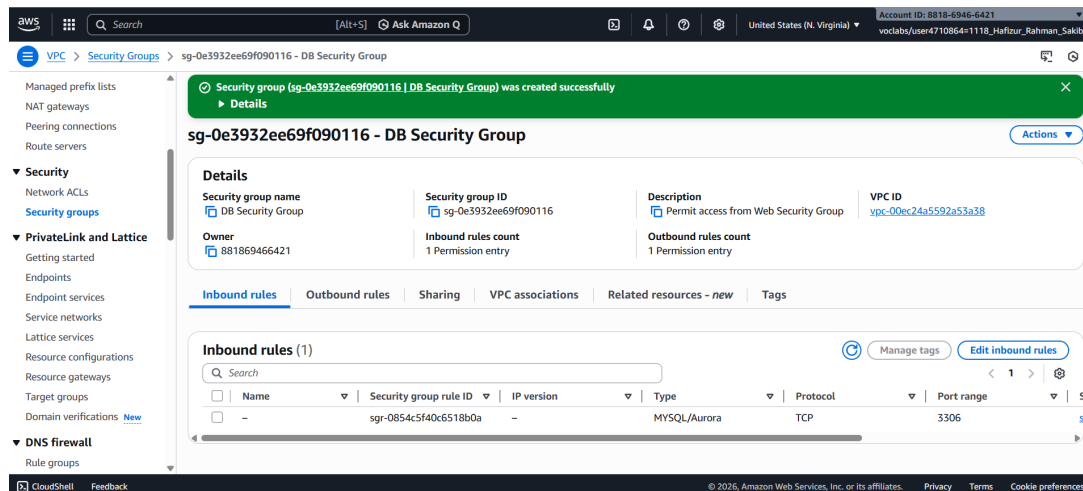


Figure 2: Inbound rules in DB Security Group allowing MySQL access from Web Security Group

This screenshot shows the rule was added correctly so only the web server could connect.

Task 2: Creating a DB Subnet Group

In the RDS console I created a DB Subnet Group named DB-Subnet-Group in Lab VPC. I picked private subnets from us-east-1a and us-east-1b.

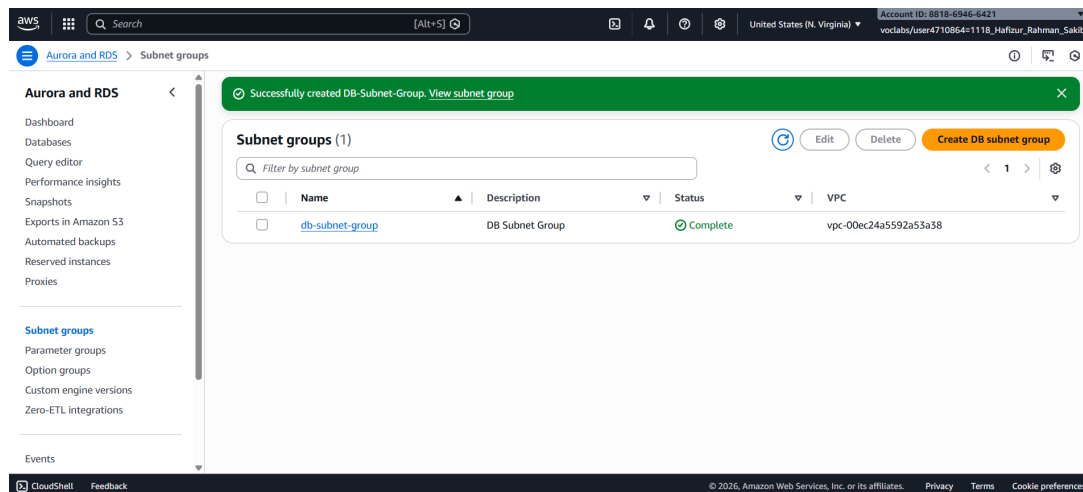


Figure 3: DB Subnet Group with subnets from two Availability Zones

This confirmed the group was set up to support Multi-AZ.

Task 3: Creating an Amazon RDS DB Instance

I created a MySQL database with these settings:

- Name: lab-db
- Engine: MySQL
- Type: Dev/Test
- Deployment: Multi-AZ
- Instance: db.t3.micro
- Storage: 20 GB General Purpose SSD
- Username: main
- Password: lab-password
- Database name: lab
- VPC: Lab VPC
- Security group: DB Security Group

I turned off extra monitoring, backups, and encryption so it would finish faster. After waiting a few minutes, it showed Available and I copied the endpoint.

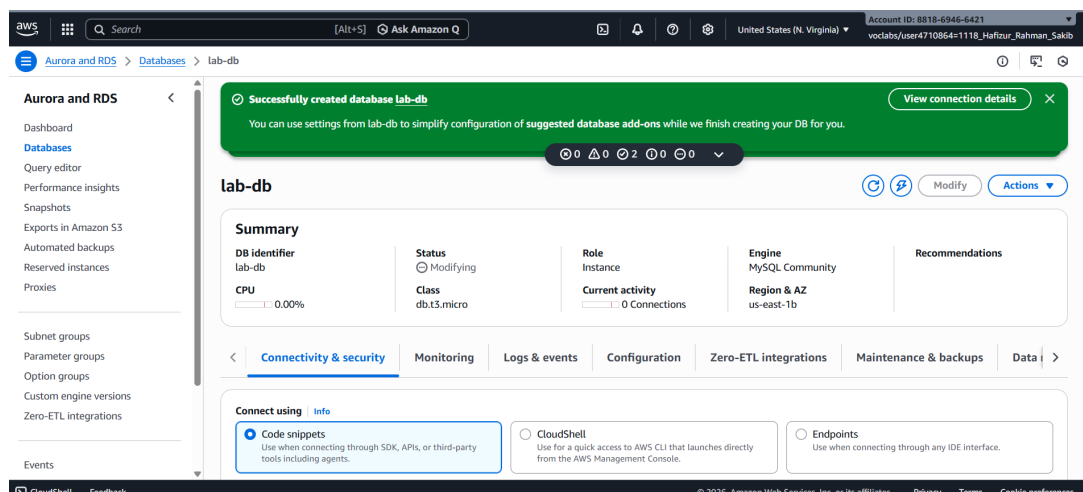


Figure 4: RDS database lab-db with Available status and Multi-AZ turned on

This screenshot shows the database was up and ready.

Task 4: Interacting with the Database Using the Web App

I opened the web server IP in a browser and went to the RDS part of the app. I filled in the RDS endpoint, database name lab, username main, password lab-password, and clicked Submit. The Address Book appeared. I added some contacts, changed one, and deleted another to test it.

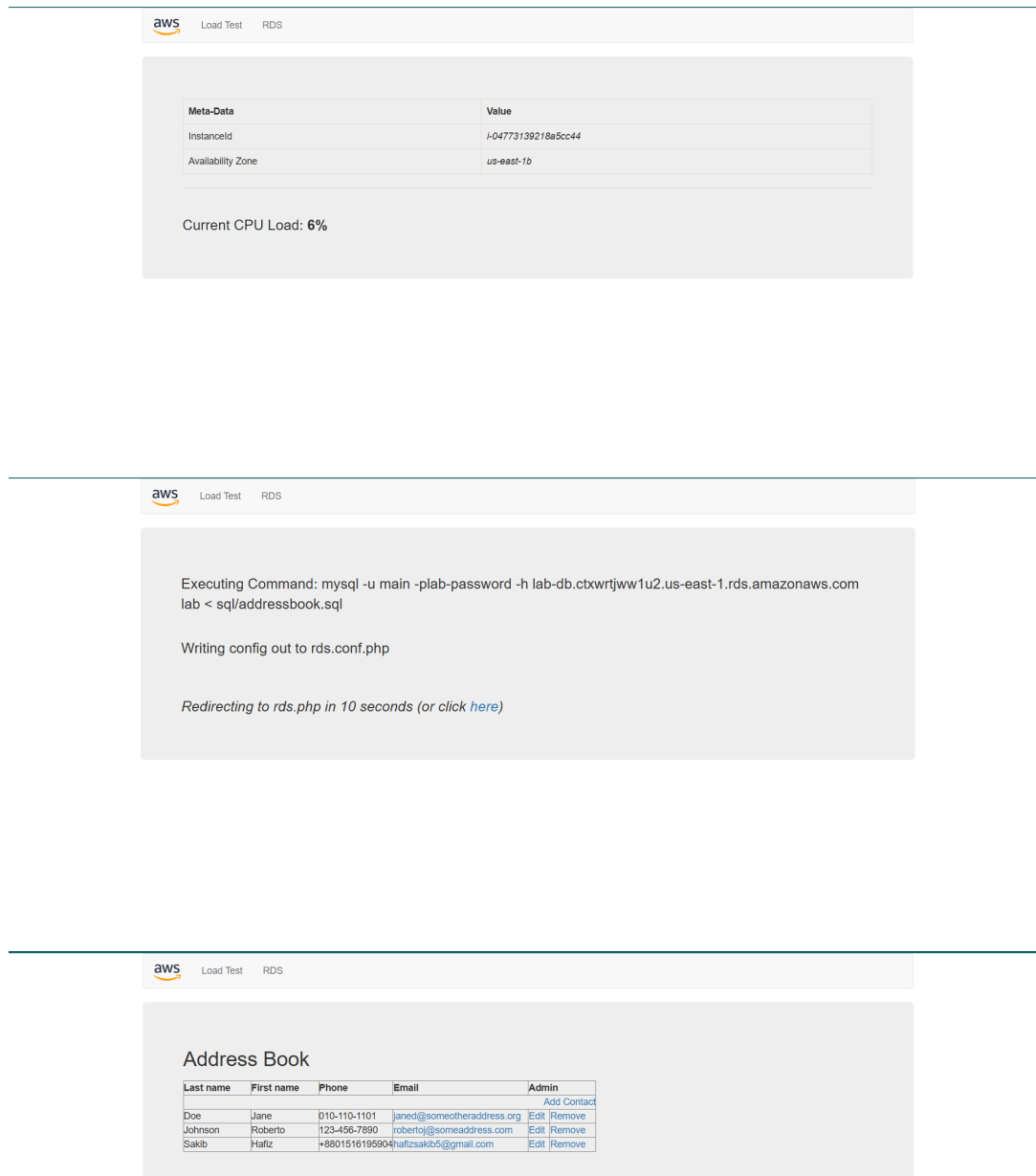


Figure 5: Address Book app displaying contacts saved in the RDS database

This last screenshot shows the app working and saving data properly.

Results Summary

All parts were done correctly:

- Made DB Security Group
- Created DB Subnet Group in two AZs
- Started Multi-AZ RDS MySQL instance lab-db
- Connected the web app to the database
- Added, edited, and deleted contacts successfully

Conclusion

This lab showed me how Amazon RDS makes setting up and running a database much simpler than doing it manually on an EC2 instance. I liked figuring out how to use security groups and subnet groups to keep things secure. The Multi-AZ part was interesting because it automatically keeps a backup copy in another zone. The best part was opening the web app and seeing it actually save contacts to the database I just made. It helped me understand why managed database services are so useful in real projects.